Case Study 7: Text Compression Using Character Runs

Activity sheet case study #35 | Concepts: Strings, loops, type conversion, operators, conditional statements, computational thinking

1. Problem Statement

Encode a string by replacing each run of consecutive identical characters with the character followed by the run length (for example aaabb becomes a3b2). Compare the original and encoded lengths and state clearly when the encoding is not beneficial.

2. Objectives

- Traverse a string with an index loop and compare neighbouring characters.
- Build a result string by concatenation and type conversion (int to str).
- Compare lengths and compute a percentage saving.
- Handle edge cases: empty string, no repeats and equal lengths.

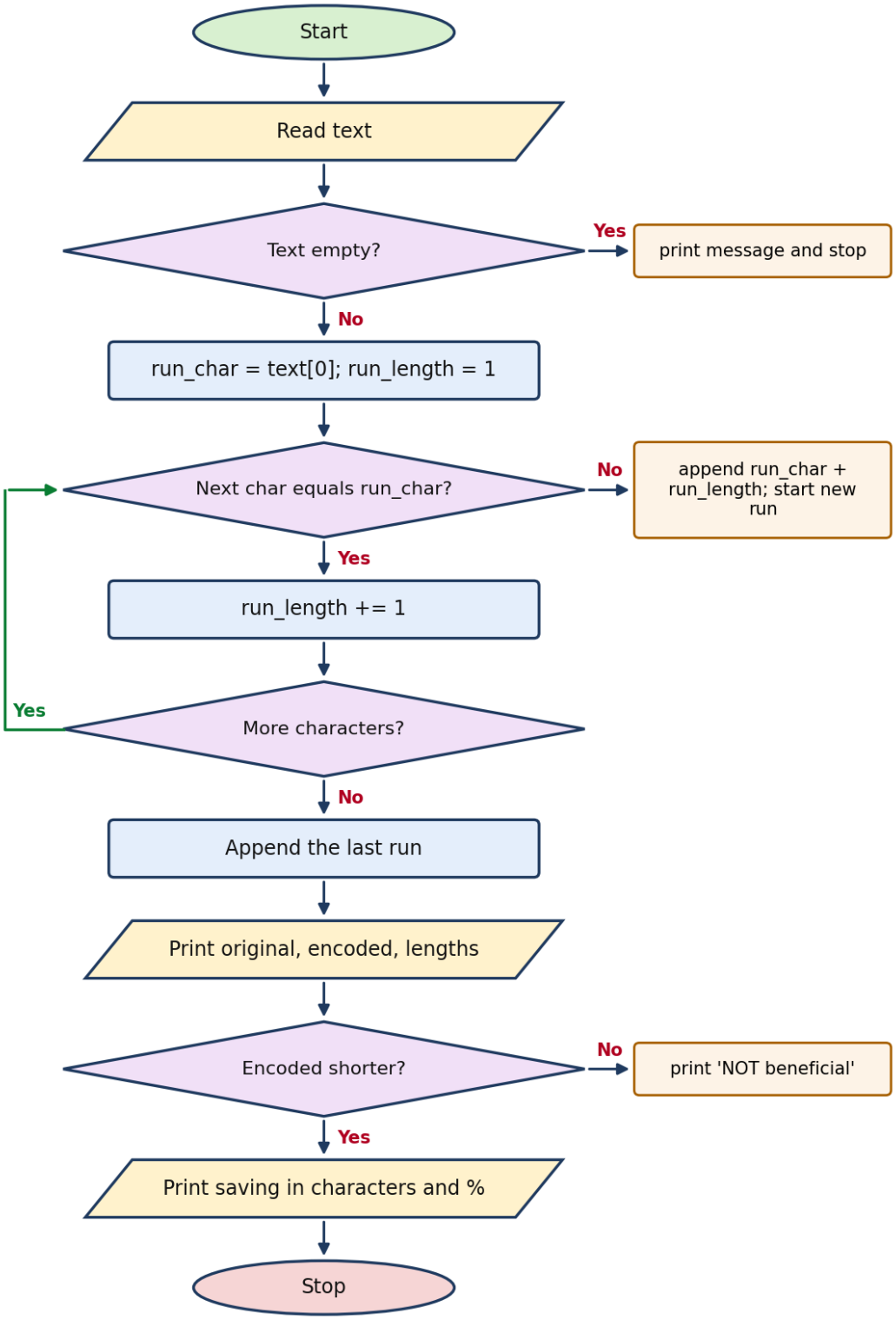
3. Assumptions

- Run-length encoding is case-sensitive: 'A' and 'a' are different characters.
- Each run is written as character + decimal count, including runs of length 1.
- Encoding is beneficial only when the encoded string is strictly shorter than the original.
- The text is read from the keyboard and may contain any characters.

4. Algorithm

1. Read the string.
2. If it is empty, print a message and stop.
3. Set run_char to the first character and run_length to 1.
4. For each following character: if equal to run_char, increase run_length; otherwise append run_char and run_length to the encoded string and start a new run.
5. After the loop append the last run.
6. Print the original, encoded string and both lengths.
7. If encoded length < original length, print the saving in characters and percent; otherwise print that encoding is not beneficial.

5. Flowchart



6. Pseudocode

```
START
  READ text
  IF LENGTH(text) = 0 THEN PRINT "empty" ; STOP
  encoded <- "" ; run_char <- text[0] ; run_length <- 1
  FOR i = 1 TO LENGTH(text) - 1
      IF text[i] = run_char THEN
          run_length <- run_length + 1
```

```
        ELSE
            encoded <- encoded + run_char + STRING(run_length)
            run_char <- text[i] ; run_length <- 1
    END FOR
    encoded <- encoded + run_char + STRING(run_length)
    PRINT text, encoded, both lengths
    IF LENGTH(encoded) < LENGTH(text) THEN
        saved <- LENGTH(text) - LENGTH(encoded)
        PRINT saved and saved / LENGTH(text) * 100
    ELSE
        PRINT "Encoding is NOT beneficial"
STOP
```

7. Python Source Code

```
# Case Study 35: Text Compression Using Character Runs
text = input("Enter a string to compress: ")

if len(text) == 0:
    print("Nothing to compress: the string is empty.")
else:
    encoded = ""
    run_char = text[0]
    run_length = 1
    for i in range(1, len(text)):
        if text[i] == run_char:
            run_length = run_length + 1
        else:
            encoded = encoded + run_char + str(run_length)
            run_char = text[i]
            run_length = 1
    encoded = encoded + run_char + str(run_length)   # last run

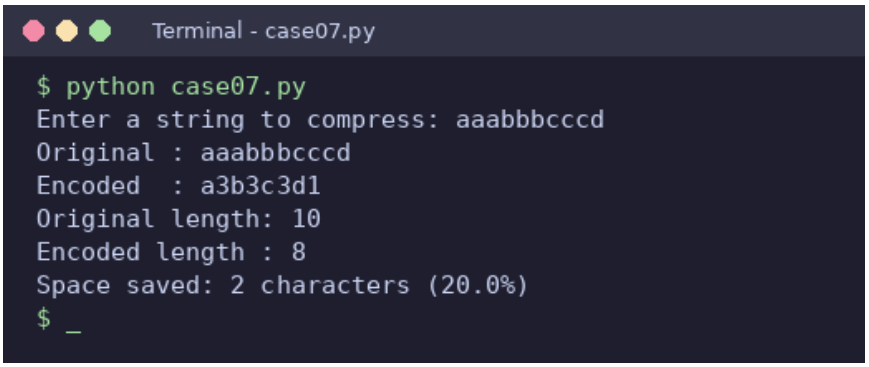
    print("Original :", text)
    print("Encoded  :", encoded)
    print("Original length:", len(text))
    print("Encoded length :", len(encoded))
    if len(encoded) < len(text):
        saved = len(text) - len(encoded)
        percent = saved / len(text) * 100
        print("Space saved: {} characters ({:.1f}%)".format(saved, percent))
    else:
        print("Encoding is NOT beneficial (encoded string is not shorter).")
```

8. Test Cases

#	Test case	Input / data	Expected result	Observed output (actual)	Status
1	Typical repeated text	aaabbbcccd	a3b3c3d1, 20 % saved	`Encoded : a3b3c3d1 | Space saved: 2 characters (20.0%)`	**PASS**
2	No repeated characters	abcd	Encoding doubles the length	`Encoded : a1b1c1d1 | Encoding is NOT beneficial (encoded string is not shorter).`	**PASS**
3	Long single run	aaaaaaaaaaaa (12 a's)	Multi-digit count; 75 % saved	`Encoded : a12 | Space saved: 9 characters (75.0%)`	**PASS**
4	Empty string	(just Enter)	Handled gracefully	`Nothing to compress: the string is empty.`	**PASS**
5	Single character	x	Longer after encoding	`Encoded : x1 | Encoding is NOT beneficial (encoded string is not shorter).`	**PASS**
6	Case sensitivity	AAaa	Equal lengths are not beneficial	`Encoded : A2a2 | Encoding is NOT beneficial (encoded string is not shorter).`	**PASS**
7	Spaces and symbols	!!! ???	Any character can form a run	`Encoded : !3 3?3`	**PASS**

9. Output Screenshot

Terminal output of `python case07.py` using the default data:



10. Conclusion

The program compresses a string in a single pass and honestly reports when run-length encoding makes the text longer, which is the key limitation of the technique. It practises string indexing, concatenation, type conversion and boundary handling for empty and single-character input.